

SPI Master-Slave Communication Using Verilog

VLSI Internship Project Report

Name: Vasamshetti Meghana

Intern ID: CITS691

Branch: ECE

Duration: 6 Weeks

Abstract

Serial Peripheral Interface (SPI) is a widely used synchronous serial communication protocol employed in embedded systems, microcontrollers, processors, sensors, memory devices, and FPGA-based applications for reliable high-speed data exchange. Due to its simple hardware structure, full-duplex communication capability, and low protocol overhead, SPI has become one of the most popular communication standards in modern digital systems.

This project presents the design, implementation, simulation, and synthesis of an SPI Master-Slave communication system using Verilog Hardware Description Language (HDL). The developed architecture consists of SPI Master and SPI Slave modules capable of exchanging 8-bit serial data through MOSI (Master Out Slave In) and MISO (Master In Slave Out) communication channels. The system utilizes Serial Clock (SCLK) and Chip Select (CS) signals to establish synchronization and control communication between the devices.

The SPI Master module is responsible for generating the clock signal, initiating data transfer, controlling the slave selection process, and managing serial transmission and reception. The SPI Slave module receives incoming data from the master, processes the information, and transmits response data through the MISO channel. Shift registers and finite state machine (FSM) based control logic are employed to achieve accurate serial communication and data synchronization.

Functional verification of the design is carried out using a comprehensive Verilog test-

bench. Simulation waveforms are analyzed to verify correct transmission and reception of serial data, proper clock generation, and successful synchronization between master and slave devices. Furthermore, synthesis is performed to evaluate hardware resource utilization and implementation feasibility.

The obtained results demonstrate successful SPI communication with accurate data transfer and reliable protocol operation. The project provides practical exposure to digital design methodologies, Verilog HDL coding, protocol implementation, simulation, and FPGA-oriented hardware design, thereby serving as a strong foundation for advanced communication protocol and System-on-Chip (SoC) development.

1 Introduction

The rapid growth of embedded systems, digital communication networks, and System-on-Chip (SoC) architectures has increased the demand for reliable and efficient communication protocols. Communication protocols play an important role in enabling data exchange between processors, memory devices, sensors, and peripheral components. Among the various serial communication protocols available today, the Serial Peripheral Interface (SPI) protocol is one of the most widely used due to its simplicity, high speed, and full-duplex communication capability.

SPI is a synchronous serial communication protocol originally developed by Motorola for communication between microcontrollers and peripheral devices. Unlike asynchronous communication protocols, SPI uses a dedicated clock signal to synchronize data transfer between communicating devices. This synchronization allows SPI to achieve higher data transfer rates with lower communication overhead.

The SPI protocol follows a master-slave architecture where the master device controls the communication process. The master generates the serial clock signal and selects the slave device using the Chip Select signal. Data transfer occurs through two separate communication channels known as MOSI (Master Out Slave In) and MISO (Master In Slave Out), enabling simultaneous transmission and reception of data.

SPI is extensively used in memory devices, sensors, ADCs, DACs, display controllers, FPGA systems, and various embedded applications. Its simple hardware implementation and high-speed operation make it suitable for modern digital systems.

In this project, SPI Master-Slave communication is designed and implemented using Verilog HDL. The design includes SPI Master, SPI Slave, clock generation logic, shift registers, and FSM-based control circuitry. Functional verification is performed through simulation, and synthesis is carried out to analyze hardware implementation feasibility. The project provides practical exposure to digital design, communication protocols, simulation methodologies, and FPGA-oriented hardware development.

2 Tools Used

The following software tools are used throughout the design, simulation, verification, and synthesis process of the SPI Master-Slave Communication system.

- Verilog HDL

Verilog Hardware Description Language (HDL) is used to design and model the SPI Master and SPI Slave modules. It provides a hardware-level description of digital circuits and allows efficient implementation of communication protocols. Verilog is widely used in FPGA and ASIC design because of its simplicity, flexibility, and support for simulation and synthesis.

- Vivado Design Suite

Vivado Design Suite is used for simulation, RTL analysis, synthesis, and implementation of the SPI communication system. It provides waveform visualization, RTL schematics, resource utilization reports, and timing analysis. Vivado helps verify the functionality of the design before hardware implementation and ensures that the design meets required performance constraints.

3 Objectives

The primary objectives of this project are:

- To understand the working principles of the Serial Peripheral Interface (SPI) protocol.
- To design SPI Master and SPI Slave modules using Verilog HDL.
- To implement serial communication using MOSI, MISO, SCLK, and Chip Select signals.
- To develop FSM-based control logic for SPI communication.
- To verify successful data transmission and reception through simulation.
- To perform synthesis and analyze hardware resource utilization.
- To gain practical exposure to digital design and FPGA implementation methodologies.
- To understand communication protocol implementation using hardware description languages.

4 SPI Protocol Overview

Serial Peripheral Interface (SPI) is a synchronous serial communication protocol widely used for data exchange between microcontrollers, processors, memory devices, sensors, and other peripheral components. It is one of the most popular communication protocols in embedded systems due to its simple architecture, high-speed operation, and full-duplex communication capability. Unlike asynchronous communication methods, SPI uses a dedicated clock signal generated by the master device to synchronize data transfer between communicating devices. The protocol follows a master-slave architecture in which the master controls the communication process while one or more slave devices respond to the master's commands. SPI communication is achieved using four primary signals: Master Out Slave In (MOSI), Master In Slave Out (MISO), Serial Clock (SCLK), and Chip Select (CS). The MOSI line is used to transmit data from the master to the slave, whereas the MISO line carries data from the slave to the master. The SCLK signal provides synchronization for data transfer, and the CS signal is used to select the desired slave device for communication. One of the key advantages of SPI is its ability to perform simultaneous transmission and reception of data, making it suitable for applications requiring high-speed and reliable communication. Due to its low communication overhead, simple hardware implementation, and efficient operation, SPI is extensively used in FPGA systems, memory interfaces, display modules, sensors, Analog-to-Digital Converters (ADCs), Digital-to-Analog Converters (DACs), and various industrial and consumer electronic applications.

The SPI communication system consists of multiple functional blocks working together to achieve reliable serial communication. The overall architecture includes the SPI Master, SPI Slave, clock generation circuitry, shift registers, and control logic.

The SPI Master serves as the central controller of the communication process. It generates the serial clock signal, controls chip selection, and manages data transmission and reception. The master initiates every communication transaction and determines when data transfer begins and ends.

The SPI Slave acts as a peripheral device that responds to commands from the master. It receives serial data through the MOSI line and transmits response data through the MISO line. The slave remains inactive until selected by the Chip Select signal.

Shift registers are used for serial-to-parallel and parallel-to-serial data conversion. These registers enable efficient handling of serial data streams during communication.

A Finite State Machine (FSM) is used to control communication sequences such as data loading, transmission, reception, and transaction completion. The FSM ensures proper synchronization between the master and slave devices.

The architecture is designed to support reliable 8-bit data communication while maintaining protocol timing requirements and synchronization constraints.

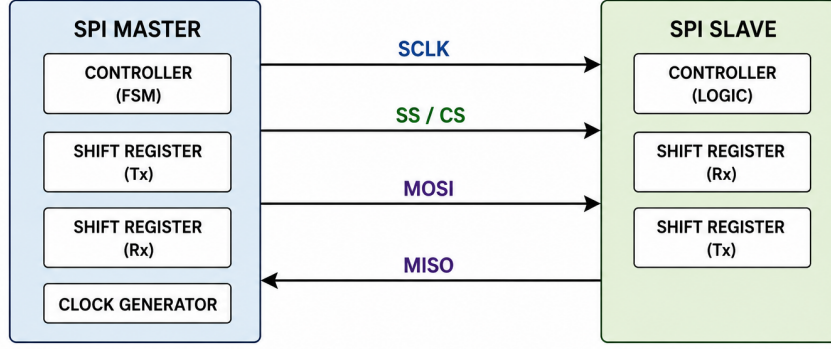


Figure 1: SPI Master Slave Architecture

5 SPI Master Design

The SPI Master is the primary controller responsible for initiating and managing the communication process between the master and slave devices. It generates the Serial Clock (SCLK) signal that synchronizes data transfer and controls the Chip Select (CS) signal used to activate the slave device. The master transmits data to the slave through the MOSI line and simultaneously receives data from the slave through the MISO line. In this project, the SPI Master is designed using Verilog HDL and consists of a clock generation unit, shift register, bit counter, control logic, and finite state machine. The shift register converts parallel input data into serial data and transmits it bit by bit on each clock cycle. The master continuously monitors the communication process and terminates the transaction after the complete data transfer is achieved. The SPI Master serves as the central coordinating unit of the communication system and ensures reliable synchronization and data exchange.

6 SPI Slave Design

The SPI Slave acts as a peripheral device that communicates with the master under the control of the Chip Select signal. The slave remains inactive until it is selected by the master through the CS signal. Once enabled, the slave receives serial data through the MOSI line and transmits response data through the MISO line. The SPI Slave is implemented using Verilog HDL and consists of shift registers, control logic, and data storage elements. The incoming serial data is shifted into the receive register and reconstructed into parallel form after all bits are received. Similarly, outgoing data is loaded into a transmit register and shifted serially through the MISO line. The slave operates synchronously with the serial clock generated by the master and ensures accurate communication by sampling data at the appropriate clock edges. The design provides reliable communication and supports efficient data exchange with the master device.

7 FSM Design

A Finite State Machine (FSM) is used to control the operation of the SPI communication system. The FSM coordinates data loading, transmission, reception, and completion of communication transactions. The design consists of multiple states including IDLE, LOAD, TRANSMIT, RECEIVE, and DONE. During the IDLE state, the system waits for a valid start signal. Once communication is initiated, the FSM transitions to the LOAD state where transmit data is loaded into the shift register. The TRANSMIT state is responsible for serially shifting data through the MOSI line while simultaneously receiving data through the MISO line. After all bits are transferred successfully, the FSM enters the RECEIVE state to complete data capture operations. Finally, the DONE state indicates successful completion of communication and returns the system to the IDLE state. The FSM ensures proper synchronization, timing control, and efficient operation of the SPI protocol.

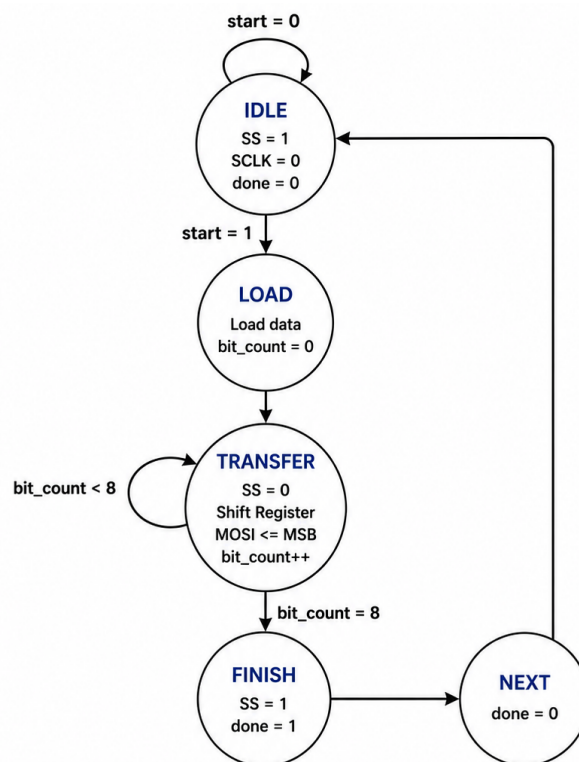


Figure 2: SPI FSM Diagram

8 Verilog HDL Implementation

8.1 SPI Master Code

```
module spi_master(  
    input clk,  
    input rst,  
    input start,  
    input [7:0] tx_data,  
  
    output reg mosi,  
    output reg sclk,  
    output reg ss,  
    output reg done  
);  
  
reg [7:0] shift_reg;  
reg [3:0] bit_count;  
  
parameter IDLE      = 2'b00;  
parameter LOAD      = 2'b01;  
parameter TRANSFER  = 2'b10;  
parameter DONE      = 2'b11;  
  
reg [1:0] state;  
  
always @(posedge clk or posedge rst)  
begin  
    if(rst)  
    begin  
        state      <= IDLE;  
        ss         <= 1'b1;  
        sclk       <= 1'b0;  
        mosi       <= 1'b0;  
        done       <= 1'b0;  
        bit_count  <= 4'd0;  
        shift_reg  <= 8'd0;  
    end  
    else  
    begin  
        case(state)
```

```

// ----- IDLE -----
IDLE:
begin
    ss      <= 1'b1;
    sclk     <= 1'b0;
    done     <= 1'b0;
    bit_count <= 4'd0;

    if(start)
        state <= LOAD;
end

// ----- LOAD -----
LOAD:
begin
    shift_reg <= tx_data;
    bit_count <= 4'd0;
    ss        <= 1'b0;
    sclk       <= 1'b0;
    state      <= TRANSFER;
end

// ----- TRANSFER -----
TRANSFER:
begin
    sclk <= ~sclk;

    if(sclk == 1'b0)
    begin
        mosi <= shift_reg[7];
        shift_reg <= shift_reg << 1;

        if(bit_count == 4'd7)
        begin
            state <= DONE;
        end
        else
        begin
            bit_count <= bit_count + 1'b1;
        end
    end
end
end

```



```

        // ----- DONE -----
DONE:
begin
    ss    <= 1'b1;
    sclk  <= 1'b0;
    done  <= 1'b1;
    state <= IDLE;
end

    default:
        state <= IDLE;

    endcase
end
end

endmodule

```

8.2 SPI Slave Code

```

module spi_slave(
    input sclk,
    input ss,
    input rst,
    input mosi,

    output reg [7:0] rx_data
);

reg [7:0] shift_reg;
reg [3:0] count;

always @(posedge sclk or posedge rst)
begin
    if(rst)
    begin
        shift_reg <= 8'b0;
        rx_data    <= 8'b0;
        count      <= 0;
    end
    else if(!ss)

```

```

        begin
            shift_reg <= {shift_reg[6:0], mosi};
            count <= count + 1;

            if(count == 7)
                begin
                    rx_data <= {shift_reg[6:0], mosi};
                    count <= 0;
                end
            end
        end
    end

end

endmodule

```

8.3 Top Module

```

module spi_top(
    input clk,
    input rst,
    input start,
    input [7:0] tx_data,

    output [7:0] rx_data,
    output done
);

wire mosi;
wire sclk;
wire ss;

spi_master MASTER(
    .clk(clk),
    .rst(rst),
    .start(start),
    .tx_data(tx_data),
    .mosi(mosi),
    .sclk(sclk),
    .ss(ss),
    .done(done)
);

spi_slave SLAVE(

```

```

        .sclk(sclk),
        .ss(ss),
        .rst(rst),
        .mosi(mosi),
        .rx_data(rx_data)
    );

endmodule

```

9 Testbench Design

```

`timescale 1ns/1ps

module spi_tb;

    reg clk;
    reg rst;
    reg start;
    reg [7:0] tx_data;

    wire [7:0] rx_data;
    wire done;

    spi_top DUT(
        .clk(clk),
        .rst(rst),
        .start(start),
        .tx_data(tx_data),
        .rx_data(rx_data),
        .done(done)
    );

    always #5 clk = ~clk;

    initial
    begin
        clk = 0;
        rst = 1;
        start = 0;
        tx_data = 0;

        #20 rst = 0;
    end

```

```

// Test 1
tx_data = 8'hAA;
start = 1;
#10 start = 0;
#250;

// Test 2
tx_data = 8'h55;
start = 1;
#10 start = 0;
#250;

// Test 3
tx_data = 8'hF0;
start = 1;
#10 start = 0;
#250;

// Test 4
tx_data = 8'h0F;
start = 1;
#10 start = 0;
#250;

// Test 5
tx_data = 8'hFF;
start = 1;
#10 start = 0;
#250;

// Test 6
tx_data = 8'h33;
start = 1;
#10 start = 0;
#250;
end

endmodule

```

The testbench verifies SPI communication between master and slave modules. Different test cases are applied to validate data transmission and reception.

10 Waveform Analysis

Simulation waveforms are used to verify the correct functionality of the SPI Master-Slave communication system designed in Verilog HDL. The waveforms confirm proper synchronization between the master and slave, correct serial data transmission, and accurate reception of data.

The SPI protocol is validated based on the following observations:

- Correct MOSI transmission from Master to Slave
- Correct MISO reception from Slave to Master
- Proper Serial Clock (SCLK) generation by Master
- Accurate Chip Select (SS) control signal operation
- Successful 8-bit serial data transfer without errors

The simulation results confirm that the SPI communication operates correctly in full-duplex mode and follows the expected timing behavior defined by the FSM control logic.

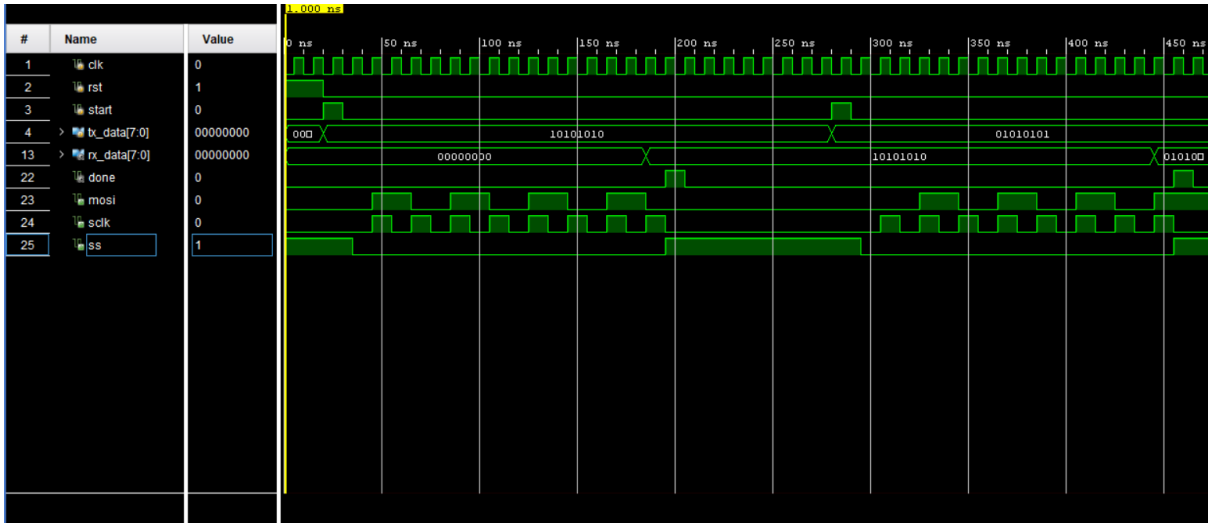


Figure 3: SPI Simulation Waveform

11 RTL Schematic

The RTL schematic represents the synthesized hardware structure of the SPI Master-Slave system. It shows the internal connections between registers, combinational logic, FSM control unit, and data paths used for serial communication.

The schematic is generated after synthesis in Vivado Design Suite and helps in understanding the hardware implementation of the Verilog design.

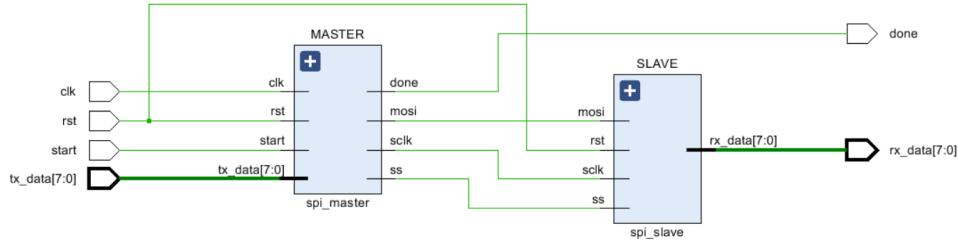


Figure 4: RTL Schematic of SPI Design

12 Synthesis Results

The SPI Master-Slave design was synthesized using Vivado Design Suite to analyze hardware resource utilization and performance. The synthesis report helps in understanding how efficiently the design is implemented in FPGA hardware.

The results show that the SPI design uses minimal resources due to its simple FSM control and shift-register based architecture, making it highly suitable for embedded FPGA applications.

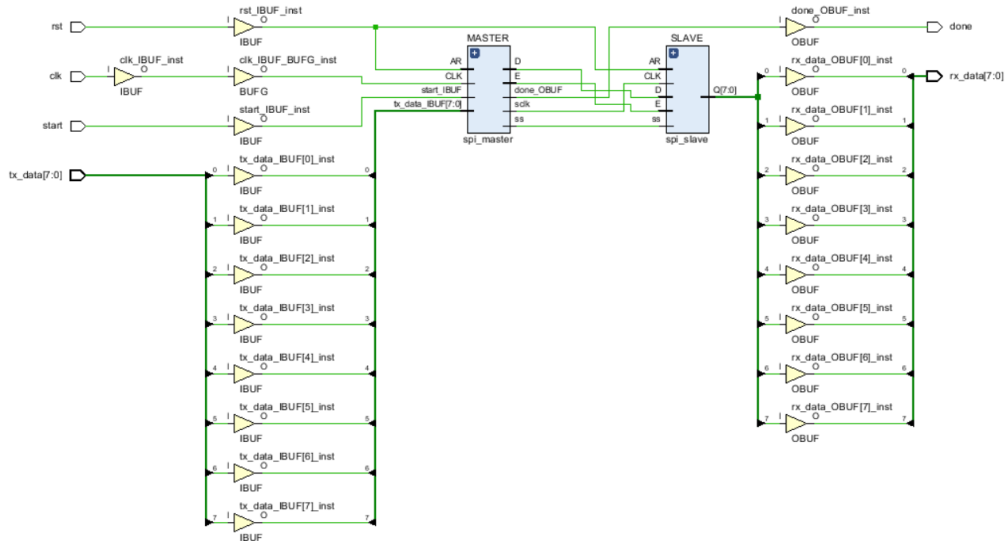


Figure 5: RTL Synthesis Result of SPI Design

Parameter	Description / Observation
Lookup Tables (LUTs)	Used for implementing combinational logic and control FSM of SPI Master and Slave modules.
Flip-Flops	Used for storing state information and shift register data during serial communication.
Slices	Represents overall FPGA resource grouping used for logic and sequential elements.
Maximum Frequency (Fmax)	Indicates the highest operating speed of the SPI design depending on FPGA constraints.
Power Consumption	Low power usage due to simple serial architecture and minimal combinational logic.

Table 1: Detailed SPI Synthesis Report

The synthesis results confirm that the SPI design is efficient, compact, and suitable for FPGA-based digital systems.

13 Results

The SPI Master-Slave communication system was successfully designed, implemented, and verified using Verilog HDL. The functionality of both master and slave modules was tested through simulation, and the results confirmed correct operation of the SPI protocol.

The simulation waveforms demonstrated that data transmission from the master to the slave through the MOSI line was accurate, and the response data from the slave through the MISO line was correctly received. The Serial Clock (SCLK) generated by the master ensured proper synchronization between both devices, and the Chip Select (SS) signal correctly controlled the activation and deactivation of the slave device.

The system successfully transmitted multiple 8-bit data patterns, and in all cases, the received output matched the transmitted input, confirming error-free communication. The Finite State Machine (FSM) based control logic ensured proper sequencing of operations such as data loading, shifting, transmission, and completion of the transaction.

Overall, the design demonstrates reliable full-duplex serial communication with proper timing control, making it suitable for FPGA-based embedded system applications.

14 Conclusion

The SPI Master-Slave communication system was successfully designed, implemented, and verified using Verilog HDL. The project demonstrates a complete working model of the SPI protocol with proper synchronization between master and slave devices.

The simulation results confirm that serial data transmission and reception are performed accurately using MOSI and MISO lines. The Serial Clock (SCLK) generated by the master ensures correct timing control, while the Chip Select (SS) signal effectively manages slave selection during communication.

The synthesis results further validate that the design is hardware-efficient and can be successfully implemented on FPGA platforms. The use of shift registers and FSM-based control logic ensures a compact and optimized design with minimal resource utilization.

Overall, this project provides practical exposure to digital system design, hardware description language (Verilog), and communication protocol implementation. The SPI design is scalable and can be extended for multi-slave communication and advanced SoC applications in the future.

15 Advantages

- High Speed
- Full Duplex Communication
- Simple Hardware
- Low Overhead

16 Applications

- Sensors
- EEPROM
- ADC/DAC
- FPGA Communication
- Embedded Systems

17 Future Scope

- **Multi-Slave SPI Communication** – Enables a single master to communicate with multiple slave devices using separate chip select signals.
- **FPGA Implementation** – Allows real-time hardware testing of the SPI design on FPGA boards for practical verification.
- **SoC Integration** – Integrates SPI module into System-on-Chip designs for internal processor and peripheral communication.

- **Higher Data Width Support** – Extends communication from 8-bit to 16-bit or 32-bit for higher data transfer efficiency.
- **SPI Mode Configuration** – Adds support for different SPI modes (Mode 0–3) based on clock polarity and phase requirements.
- **Speed Optimization** – Improves clock generation logic to achieve higher data transfer rates.
- **Low Power Design Enhancement** – Reduces power consumption for embedded and battery-operated applications.

18 References

- SPI Protocol Specification
- Verilog HDL by Samir Palnitkar
- FPGA Design Guides
- Digital Design by Morris Mano